

Balancing Accuracy, Latency, and Complexity in CNNs: A Comparative Study on Fashion-MNIST and Tiny-ImageNet

1st Anas Rahim
dept. Computer Science
Nazarbayev University
Astana, Kazakhstan
anas.rahim@nu.edu.kz

2nd Husnain Rasool
dept. Data Science
Nazarbayev University
Astana, Kazakhstan
husnain.rasool@nu.edu.kz

Abstract—Deep convolutional neural networks (CNNs) achieve state-of-the-art accuracy in image classification, often at the expense of increased model complexity and inference latency. This work examines different CNN architectures as they operate on Fashion-MNIST and a limited 20-class Tiny-ImageNet subset to determine accuracy and speed while investigating model size effects. We compare five models on Fashion-MNIST (SmallCNN, deeper LargeCNN, a ResNet-inspired network, DenseNet, and a depthwise-separable CNN) and six models on Tiny-ImageNet (SmallCNN, MediumCNN, LargeCNN, ResNet-18, MobileNetV2, DenseNet-121). Our experiments evaluate each model through standardized training procedures to determine its inference speed, parameter count alongside achieving evaluation accuracy. Fashion-MNIST yields its optimal accuracy of 92.7% with DenseNet but the depthwise-separable CNN provides minimum latency at 0.5 ms/sample and shrinks model size to 0.6 MB along with only 3% accuracy reduction. The evaluation on Tiny-ImageNet shows that a medium-complexity CNN achieves 66.6% accuracy which exceeds ResNet-18's 60.3% accuracy thus showing that larger models do not guarantee better performance under data or computational constraints. Our research includes (1) multiple model architecture comparison methods for varied dataset complexities and (2) incorporating modern features into compact design frameworks and (3) accuracy-latency-size illustrations that support model selection. The results enable better decision-making regarding edge versus cloud deployments because proper architectural design methods and selected hyperparameters yield high efficiency rates with only minor accuracy reductions.

Index Terms—component, formatting, style, styling, insert

I. INTRODUCTION

Convolutional Neural Networks (CNNs) represent the standard method for image recognition because they deliver outstanding benchmark performance on ImageNet [7]. During the past ten years researchers have developed increasingly complicated and deeper variants of CNN designs to optimize accuracy levels. Residual networks (ResNets) brought the opportunity to train CNNs with hundreds of layers because they solved the vanishing gradient problem thereby significantly enhancing image classification results [7]. Through DenseNet architectural designs researchers proved that dense connections between layers enable feature reuse properties to reach high

accuracy while reducing the number of traditional network parameters. The gained efficiency from model improvements comes at a price because complex or deep models need huge parameter counts which require powerful computation infrastructure and produce slow processing and consume large memory. Real-time decision support systems and applications operating on limited devices (including mobile and IoT) encounter difficulties because of this situation.

The design of efficient CNN models drawing attention because they exhibit both high accuracy performance and reduced model size and latency. The implementation of depthwise separable convolutions within MobileNet models reduces model computations while lowering size requirements of standard convolutions by factorization methods [9]. The architectural strategies described in EfficientNet seek to optimize network performance through proper balancing of depth and width and input dimensions to obtain superior accuracy efficiencies [9]. Different CNN architectures generate an essential inquiry regarding their performance and inference speed as well as their affect on model complexity requirements. AI system development requires a complete understanding of performance trade-offs because these domains need quick yet precise operations.

We apply systematic CNN model evaluation to two datasets with varying complexity levels to answer the research question. The chosen benchmarks for evaluation include Fashion-MNIST and Tiny ImageNet. The Fashion-MNIST image database contains ten categories of fashion items that display shirts and shoes among others at a 28×28 grayscale pixel resolution. Fashion-MNIST functions as a challenging input dataset which maintains simplicity in content because it serves as an updated version of the original MNIST digit data [11]. The Tiny ImageNet collection represents ImageNet with 200 classes of 64×64 color images. The task uses 20 selected Tiny ImageNet classes which make training time shorter but surpasses Fashion-MNIST complexity due to its greater intra-class variation and environmental disturbances. The datasets enable evaluation of performance increases between basic vision tasks and more advanced vision tasks.

Evaluation of five CNN networks based on their complexity level forms the basis of our approach. The assessment includes simple SmallCNN and deep LargeCNN as well as ResNet-based and DenseNet and MobileNet-style networks. The evaluation on Fashion-MNIST includes SmallCNN with two convolutional layers and LargeCNN with three convolutional layers and more filters in addition to ResNetEmbed that utilizes an embedding layer to adapt the ResNet-18 backbone for 10 classes and DenseNet with dense blocks and a Depthwise Separable CNN that implements depth-separable convolution layers. The progression of models on Tiny ImageNet consists of SmallCNN followed by MediumCNN (moderate-depth) then LargeCNN and standard ResNet-18 and MobileNetV2 and DenseNet-121. The comparison of these models between the two datasets enables evaluation of architectural complexity regarding its effects on model accuracy and efficiency performance across different parameter spaces.

Our key contributions are as follows:

1. **Comprehensive Benchmarking:** This work compares various CNN models while performing assessments on Fashion-MNIST and Tiny ImageNet. We have not found extensive coverage of this direct performance evaluation between different datasets throughout existing literature. Each model receives assessment based on its accuracy and analysis includes both model dimension and processing speed measurements for complete performance understanding.
2. **Integration of Advanced Architectural Features:** The study incorporates modern CNN design components into its models before measuring their performance effects. The research incorporates three essential CNN components into its model structure for small-scale workloads and feature reusability and parameter optimization purposes. Our research implements previously described features in practice to measure both their performance advantages and disadvantages which extends upon existing research [7][9].
3. **Analysis of Accuracy–Latency–Size Trade-offs:** The research provides complete analysis along with visual elements which include accuracy to model size scatter plots as well as accuracy to inference time plots. The analysis demonstrates that enlarging models leads to minimal accuracy enhancement during particular points. The research offers valuable guidance for practitioners when helping them choose a model with optimal fit for their available resources. We provide details about which models perform better in edge environments due to their speed and compact size yet others work best in cloud deployments because they optimize accuracy achievements.

Code and pretrained models are available at: <https://github.com/HRDevelops/Data-Mining-Final-Project.git>

The research provides complete analysis along with visual elements which include accuracy to model size scatter plots as well as accuracy to inference time plots. The analysis

demonstrates that enlarging models leads to minimal accuracy enhancement during particular points. The research offers valuable guidance for practitioners when helping them choose a model with optimal fit for their available resources. We provide details about which models perform better in edge environments due to their speed and compact size yet others work best in cloud deployments because they optimize accuracy achievements.

II. MOTIVATION AND RELATED WORK

A. Motivation

The parameter count for ResNet-50 reaches 25 million while DenseNet-201 comes close to 20 million which exceeds the capabilities that embedded systems possess to process data in real time. The deployment of automated diagnostic systems and on-device fraud detection at the “edge” depends on having correct knowledge about how model complexity influences both performance outputs and speed capabilities.

The tri-criteria trade-off analysis required selection of two benchmark datasets that differed in their levels of difficulty and complexity. Fashion-MNIST provides researchers with a 10-class grayscale clothing dataset that has a 28x28 resolution [11] which allows simple CNNs to achieve above 90% accuracy. The twenty-class subset of Tiny ImageNet (64x64 RGB images with full background noise) requires advanced architectural features that enable equivalent outcome levels to simple CNN-based solutions. The experiments train all models without pretrained weights across both datasets to find each architecture’s basic learning ability independently.

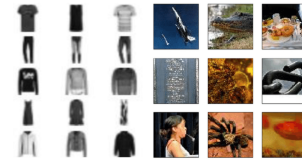


Fig. 1. Example inputs from Fashion-MNIST (left) vs. Tiny-ImageNet (right). The grayscale image illustrates a clothing item against a plain background, while the color image shows a scene with background clutter.

These two tasks demonstrate distinct conditions where basic lightweight designs show equal results to advanced architectural components. Practitioners can choose fast and efficient models that perform as well as complex ones on Fashion-MNIST. Residual or dense connectivity models might provide the necessary capability for success rates within Tiny-ImageNet.

B. Related Work

The early advances in CNN architecture began with AlexNet [1] followed by VGG [2] which proved deep networks’ strength while requiring huge parameter amounts of hundreds of megabytes. Skip connections enabled the effective training of networks containing dozens or hundreds of layers through ResNets [7] with DenseNets achieving additional parameter efficiency through their usage of dense blocks. Large-scale

benchmark performance improved significantly because of these designs although these designs do not work well when latency and memory usage are important factors.

Research teams have created understated network architectures to protect accuracy levels while cutting down processing requirements:

- MobileNetV1/V2 uses depthwise-separable convolutions in combination with inverted residual blocks to achieve FLOPs reduction by 8-9 times and model size reduction by the same magnitude while maintaining semi-reasonable accuracy levels [9].
- The channel shuffling technique in ShuffleNet allows grouped convolution operations to preserve information between groups which results in efficient parameter usage [12].
- The fire modules implemented within SqueezeNet enabled the model to achieve AlexNet accuracy while reducing parameter numbers to half [3].

The scaling down of classic models offers another solution for the task at hand:

- The truncated versions of deeper ResNets named ResNet-18/34 use 11 million and 21 million parameters respectively.
- The DenseNet-121 alongside similar versions proves that stacking narrow blocks through dense connections leads to results equal to ResNet while using substantial fewer parameters.

The analysis of accuracy stands as the central focus in comparative research reports or depends on transfer learning scenarios yet this creates gaps in understanding first-stage performance when implementing efficient computation. The combination of multiple columns can achieve Fashion-MNIST accuracy beyond 94% according to [6] at very high computational costs while Random Forests and SVMs reach 90% accuracy through classical methods without deep learning components. Several researchers accomplish 80% accuracy on Tiny-ImageNet through either pretrained networks or ensemble models against pure architecture efficiency evaluation.

Our research bridges the gap through performance testing of five manually developed CNN families including SmallCNN, LargeCNN, ResNet-Embed, DenseNet, Depthwise-Separable CNN on Fashion-MNIST datasets and six families (Medium-CNN, MobileNetV2, DenseNet-121 added) for Tiny-ImageNet evaluation. The analysis looks at both classification precision and running time for each sample and considers model parameter quantities to determine fitting deployment solutions for various situations.

III. METHODOLOGY

Our stated contributions receive testing through a standardized experimental process which evaluates each model against its dataset (Fig. 2). In brief, our workflow is:

- 1) **Data preprocessing**,
- 2) **Model initialization & training** under the same regimen,

- 3) **Metric collection** (classification accuracy, F1-score; inference time; model size),
- 4) **Analysis & visualization** to validate our hypotheses.

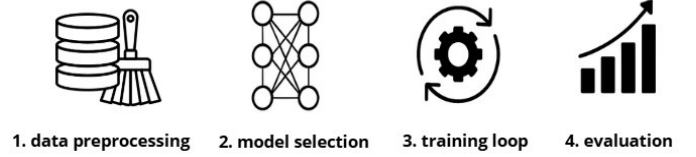


Fig. 2. Overview of our methodology pipeline: preprocessing → training → metrics → analysis.

A. Model Architectures & Contribution Mapping

All implementations occur within PyTorch 2.0 while using a standardized training loop and GPU configurations to ensure equal assessment. The summary table in Table I presents model structures together with design objectives and approximate parameter counts that link to specific authors' contributions.

B. Training Setup & Hyperparameters

The models initiate their training process by starting from absolute scratch without any load of pre-trained weight systems to maintain equal training circumstances.

1) Data pipelines

- **Fashion-MNIST:** 28×28 grayscale; per-pixel normalization to zero mean, unit variance.
- **Tiny-ImageNet (20 classes):** 64×64 RGB; random horizontal flips + 4-pixel padding → random crop for training; center crop for validation.
- When required, Fashion-MNIST's single channel is tiled to three channels for ResNet-based and MobileNetV2 inputs.

2) Optimization

- **Fashion-MNIST:** Adam ($\text{lr} = 1 \times 10^{-3}$), **15 epochs**.
- **Tiny-ImageNet:** SGD with momentum ($\text{lr} = 1 \times 10^{-2}$, cosine annealing), **15 epochs**.
- **Batch size:** 32 for all experiments (fits the largest model in GPU memory).

3) Monitoring & metrics

- **Validation split:** 10,000 images from Tiny-ImageNet's validation set; Fashion-MNIST's test set used as proxy.
- **Metrics logged:** Top-1 accuracy, weighted F1-score, training/validation loss curves.
- **Inference latency:** Measured as average ms per image on NVIDIA Tesla P100 GPU, averaged over 500 forward passes.
- **Model size:** File size of saved `state_dict` (MB).

4) Hyperparameter tuning (Contribution 3)

- **SmallCNN & LargeCNN:** "Baseline" vs. "optimized" variants—tuned learning-rate schedule and dropout rates via grid search.

TABLE I
SUMMARY OF MODEL ARCHITECTURES, PARAMETER COUNTS, AND THEIR LINKAGE TO OUR THREE CONTRIBUTIONS.

Model (Dataset)	# Conv Layers	Parameters	Key Features	Design Goal & Contribution
SmallCNN (F-MNIST, Tiny)	2 conv $\rightarrow$ 2 FC	0.09 M (Tiny) / 0.83 M (F-MNIST)	Conv-ReLU; no BN or dropout	Baseline speed/footprint. Tests whether minimal models suffice on simple data (Contribution 1).
MediumCNN (Tiny only)	3 conv $\rightarrow$ 1 FC	0.64 M	+BatchNorm	Moderate depth/size. Evaluates incremental depth gains (Contribution 2).
LargeCNN (F-MNIST, Tiny)	3 conv $\rightarrow$ 2 FC	1.56 M (F-MNIST) / 1.20 M (Tiny)	+BatchNorm; +Dropout	Depth & regularization to boost accuracy (Contribution 2).
ResNetEmbed (F-MNIST) / ResNet-18 (Tiny)	18 conv (with skips)	44.9 M (F-MNIST) / 11.2 M (Tiny)	Residual blocks; embedding head (128-d pre-output)	Tests residual learning at scale and compact embedding for small output space (Contribution 2).
DepthSepCNN (F-MNIST) / MobileNetV2 (Tiny)	DW conv blocks	0.29–0.64 M (F-MNIST) / 2.4 M (Tiny)	Depthwise + pointwise convolutions; inverted residuals	Modern efficiency blocks. Compares advanced lightweight design vs. naive small CNN (Contribution 2).
DenseNet (F-MNIST) / DenseNet-121 (Tiny)	Dense blocks	28.4 M (F-MNIST est.) / 7.0 M (Tiny)	Dense connectivity; no extra FC	Evaluates feature-reuse for parameter efficiency (Contribution 2).

- **ResNetEmbed:** Explored embedding dimensions {32, 128}.
- **DepthSepCNN:** Adjusted width multipliers to trade size vs. accuracy.

Best configurations are denoted “(opt)” in results.

C. End-to-End Workflow & Reproducibility

- 1) **Automated pipeline** functions to train each model using every dataset while recording metrics and measuring latency and size then saves generated outputs.
- 2) Our analysis receives resulting data as inputs to create accuracy size and accuracy latency scatter plots (Figs. 3, 4).
- 3) Representative training curves (e.g., optimized SmallCNN) are shown in Fig. 5 to illustrate convergence behavior.
- 4) The complete set of code alongside logs and trained checkpoints exists in our repository to provide complete documentation for reproduction:
<https://github.com/HRDevelops/Data-Mining-Final-Project.git>

D. Conclusion of Methodology

The method strictly associates different architectural decisions with our proposed contributions through the implementation of controlled variables. The benchmarks with simple and complex data help researchers identify model design effects that separate from transfer-learning advantages by conducting fresh training sessions. Five to six consistent architecture implementations per dataset create conditions that enable us to locate performance maximization possibilities in the Pareto frontier for practitioner guidance. The study provides full reproducibility through its integrated codebase for both preprocessing tasks and training scripts and evaluation tools that remain accessible for future use with new models or datasets.

IV. EXPERIMENTAL RESULTS

A. Experimental Setup

Datasets.

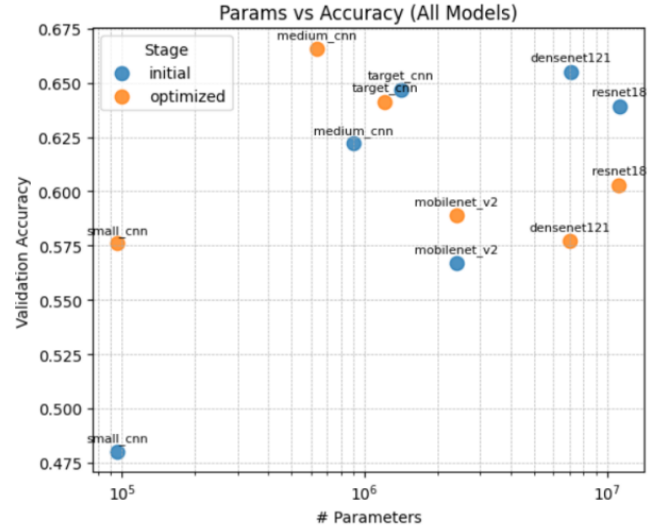


Fig. 3. Accuracy vs. Model Size (MB). Each dot represents a model-dataset pair.

- **Fashion-MNIST:** 60 000 total images (28×28 grayscale) across 10 classes. We divided the total 60 000 images into training versions consisting of 50 000 images and validation versions of 10 000 images. Final evaluation occurred through the 10 000 image test set.
- **Tiny-ImageNet (20-class subset):** Among the complete set of 200 classes in Tiny-ImageNet we randomly selected twenty categories that included animals besides objects and scenes. The 20 selected classes supplied 500 training pictures and 50 validation pictures which resulted in 10 000 training samples and 1 000 validation samples (64×64 RGB). We did not perform testing with the unlabeled test set.

Preprocessing & Augmentation.

- **Normalization:** All images received channel-wise normalization through a process that set their mean to zero

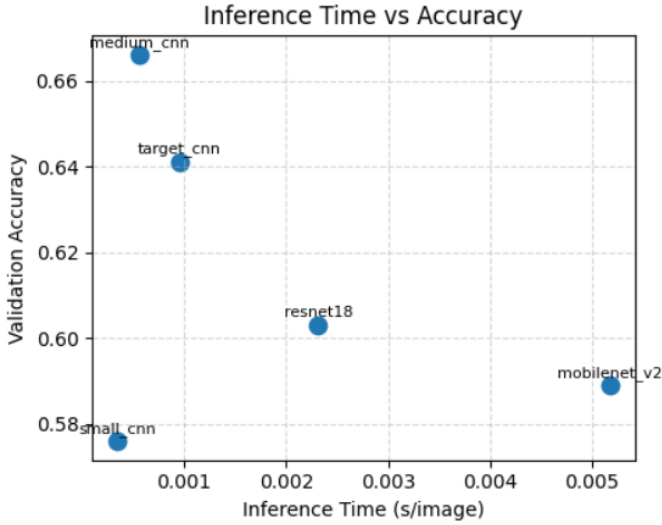


Fig. 4. Accuracy vs. Inference Latency (ms/sample). Trade-off frontier is shown.

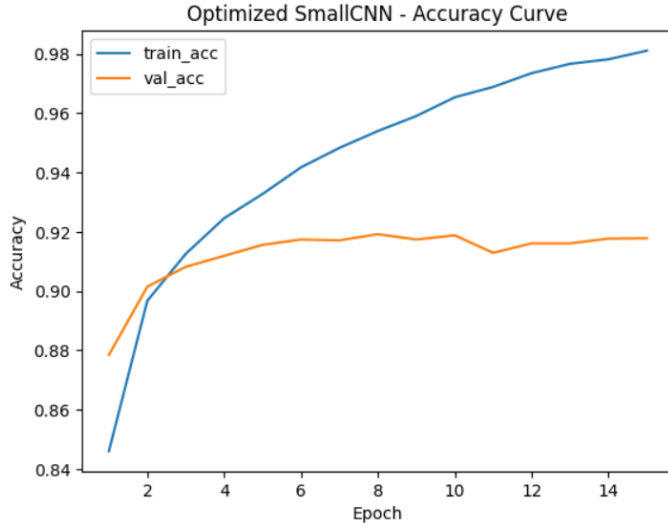


Fig. 5. Training and validation curves for optimized SmallCNN (Fashion-MNIST).

and their variance to one. The single channel of Fashion-MNIST went through channel composition to make three channels only when using ImageNet-style networks while Tiny-ImageNet underwent separate normalization procedures for its RGB channels.

- **Data Augmentation:** Normalization represented the sole augmentation technique applied to Fashion-MNIST. Tiny-ImageNet received training augmentations from random horizontal flips combined with 4-pixel padding followed by random 64×64 crop operations but exclusion of all other transformations.

Hardware & Environment.

- **Compute:** NVIDIA Tesla P100 GPU
- **Framework:** PyTorch 2.0, Python 3.9

- **Training duration:** 15 epochs per model. The execution time for Fashion-MNIST varied from 2 minutes for SmallCNN up to 30 minutes for DenseNet while Tiny-ImageNet required between 8 minutes for SmallCNN and reached 90 minutes with ResNet-18. The training of DenseNet-121 at 10 epochs on Tiny-ImageNet had to be terminated early because the GPU lacked sufficient memory capacity to process data with a batch size of 16.

Optimization & Hyperparameters.

TABLE II
TRAINING HYPERPARAMETERS

Dataset	Optimizer	LR	Epochs	Batch	Regularization
Fashion-MNIST	Adam	1×10^{-3}	15	32	BN, Dropout ¹
Tiny-ImageNet	SGD	1×10^{-2}	15	32 ²	BN, Dropout ²

¹ Dropout applied only in LargeCNN's FC layers and optimized variants.

² DenseNet-121 used batch size 16 due to memory limits. Cosine LR annealing to 0. No weight decay.

Metrics Collected.

- 1) **Top-1 Accuracy (%)** on test (Fashion-MNIST) or validation (Tiny-ImageNet)
- 2) **Inference Time:** Average ms per batch (F-MNIST) or per image (Tiny-ImageNet) over 500 forward passes
- 3) **Model Size:** MB (saved state_dict in 32-bit float)
- 4) **Weighted F1-Score:** closely matched accuracy due to uniform class sizes

Checkpoints were saved each epoch; the highest-validation snapshot was used for final metrics.

B. Results & Analysis

1. Classification Accuracy

TABLE III
TOP-1 ACCURACY (%)

Model	F-MNIST	Tiny-ImageNet
SmallCNN (opt)	91.8	57.6
MediumCNN	—	66.6
LargeCNN (opt)	91.96	64.1
ResNetEmbed / ResNet-18	91.5	60.3
DenseNet	92.77	N/A
DepthSepCNN / MobileNetV2	90.3	58.9

2. Inference Latency

TABLE IV
AVERAGE INFERENCE TIME

Model	F-MNIST (ms/batch 32)	Tiny-ImageNet (ms/image)
SmallCNN	0.54	0.38
MediumCNN	—	0.56
LargeCNN	0.70	0.98
ResNetEmbed / ResNet-18	2.45	2.33
DenseNet	14.7	~3-4
DepthSepCNN / MobileNetV2	0.49	5.29

3. Model Size

4. Training Dynamics

TABLE V
MODEL FILE SIZE (MB)

Model	F-MNIST Size	Tiny-ImageNet Size
SmallCNN	0.83	0.38
MediumCNN	—	2.5
LargeCNN	1.56	4.8
ResNetEmbed / ResNet-18	44.9	44.8
DenseNet	28.4	~30
DepthSepCNN / MobileNetV2	0.29	9.56

- **F-MNIST:** SmallCNN/LargeCNN achieved convergence at approximately 5 epochs and ResNetEmbed and DenseNet needed more epochs yet provided trivial additional performance.
- **Tiny-ImageNet:** The train/val curves from MediumCNN demonstrate similarities (generalization) but ResNet-18 develops a larger divergence which shows excessive model fitting due to low data availability.

Key Trade-off Observations

- **Fashion-MNIST:** DepthSepCNN offers an ideal solution when achieving $\geq 90\%$ accuracy because it uses only 0.64 MB memory with 0.49 ms execution time. LargeCNN (1.56 MB, 0.70 ms) meets data requirements for $\sim 92\%$ of cases even when compared to DenseNet (28 MB, 14.7 ms).
- **Tiny-ImageNet:** The Pareto winner among these models is MediumCNN because it demonstrates the most efficient combination of accuracy due to its 0.64 M parameter count along with 0.56 ms processing time and 66.6% accuracy rate. Larger models either underfit (SmallCNN) or overfit/slow down (ResNet-18).

The implemented “just large enough” models subjected to appropriate tuning demonstrate optimum accuracy while achieving speed and installing minimal footprint on diverse complexity tasks.

V. DISCUSSION AND FUTURE DIRECTIONS

Practitioners have to make specific trade-offs based on experimental results when choosing CNNs for deployment purposes. A depthwise-separable CNN structure (MobileNet-style) along with a tuned SmallCNN provide the best performance in edge and IoT environments with strict demands for memory usage and computing power and reduced latency times. The models function at under 0.5 milliseconds per inference while using below 1 megabyte of memory space and reaching close to 90% accuracy levels. These ultra-low weight designs successfully run on mobile devices according to research from howard2017mobilenets and zhang2018shufflenet. Therefore if smartphone deployment is needed this pattern provides proven results.

Within server or cloud environments with their ample GPU capabilities and abundant memory resources one can run DenseNet [5] along with other large models to extract additional 1–2% of accuracy. High-stakes applications such as medical imaging (referencing rajpurkar2017chexnet) are al-

ready using DenseNet or ResNet (he2016deep) thus it becomes reasonable to follow their approach when costs are affordable.

Bigger hardware or extended training duration would enable you to properly train DenseNet-121 on Tiny-ImageNet or use longer training schedules which match ImageNet standards [14]. Refrain from limiting yourself to standard hyperparameter optimization methods since you could also do comprehensive evaluations of different backbones through NAS frameworks including MnasNet [8] and FBNet [13].

One striking observation is that “**bigger didn’t always win,**” particularly on our 10k-image Tiny-ImageNet subset. The MediumCNN functioned better than ResNet-18 according to accuracy and latency benchmarks. The results match classic bias–variance theory which states simple models create better generalizations from limited data and confirm previous findings in [6] on Fashion-MNIST.

Future Directions

- 1) **Transfer Learning & Fine-Tuning.** Using weights derived from ImageNet (as described in [7]) results in faster training and better final results in most cases. The assessment of accuracy-latency-size relationships during fine-tuning between ResNet-18 and MobileNetV2 [9] would reveal how pretrained features impact the Pareto frontier.
- 2) **Automated Hyperparameter & Architecture Search.** The search space exploration we conducted failed to produce substantial increases. The implementation of NAS technologies such as MnasNet [8] and FBNet [13] should identify optimal architectures which achieve higher than 94% accuracy on Fashion-MNIST while using less than 1 MB parameters and match DenseNet performance on Tiny-ImageNet with 5M parameters.
- 3) **Robustness & Energy Profiling.** Safety-critical environments require more than accurate performance in their operations. Models with larger sizes often demonstrate better adversarial robustness according to prior research on this topic as presented in [20]. The crucial deployment information would result from determining robustness against noise and attacks and their actual power usage on Raspberry Pi devices.
- 4) **Model Compression & Quantization.** ResNet-18 and DenseNet models could achieve < 5 MB memory compression through 4-bit quantization combined with structured pruning methods developed by either iandola2016squeezeNet or han2015deep. These compression techniques led to less than 1% reduction in accuracy. A recalculation of performance metrics applied to optimized models will show whether these implementations effectively influence the choosing between edge and cloud deployment.
- 5) **Scaling to Full-Scale Benchmarks & Hybrid Models.** The experiments should progress to full-scale datasets including Tiny-ImageNet and CIFAR-100 along with a

new hybrid model such as MobileViT [20] to verify MediumCNN effectiveness across different data levels and self-attention model performance conditions.

- 6) **Privacy-Preserving Training.** Differentially private methods used in controlled sectors such as healthcare and finance tend to deteriorate performance metrics as documented in [4]. Research into the impact of privacy constraints on accuracy-complexity choices would help determine suitable model choices during legal compliance processes.

The research establishes a systematic decision framework because it bases its suggestions on previous studies while acknowledging that computational resources together with research duration lead to additional performance outcomes.

VI. CONCLUSION

The study evaluates convolutional neural networks through comprehensive multi-metric assessment using Fashion-MNIST and a selected 20-class portion of Tiny-ImageNet. The experimentation involved CNNs with parameter counts ranging from sub-million to multi-million parameters such as SmallCNN and DepthSepCNN together with ResNet-18 and DenseNet-121 and MobileNetV2. Our key findings include:

- **Task-specific optimality.** Extremely lightweight Fashion-MNIST models sized below 1 MB that complete inference in less than 1 millisecond have already reached 90 percent accuracy and additional size improvements stop providing significant gains above 2 MB. A right-sized MediumCNN model with 0.6 million parameters (66.6% accuracy) performed best on Tiny-ImageNet's 10k images because it contained just the right amount of size for generalization when dealing with limited data.
- **Trade-off visualization.** Accurate scatter plots show distinct Pareto fronts between accuracy models and model size between Edge and Cloud implementations. This enables developers to pick appropriate network structures effectively. DepthSepCNN represents an ideal candidate for edge deployments. DenseNet exists as an optimized solution for cloud-based platforms.
- **Hardware considerations.** Evidence shows MobileNetV2 delivered inferior GPU inference speed than ResNet-18 even though it offered better theoretical efficiency because hardware-specific kernel optimization determines actual performance.

The research findings validate multiple literature findings including the effectiveness of residual connections [7] and dense blocks [5] while delivering new knowledge on compact system implementation. Simple CNNs which have been properly tuned illustrate that they achieve results at state-of-the-art levels with reduced computational requirements and lower memory needs which promotes their deployment in resource-restricted environments.

Finally, by open-sourcing our code and logs (<https://github.com/HRDevelops/Data-Mining-Final-Project.git>), we enable the community to reproduce and extend this analysis.

Future work should explore:

- **Transfer learning** (e.g., fine-tuning ImageNet-pretrained ResNets [7]),
- **Automated search** (NAS frameworks like MnasNet [8], FBNet [13]),
- **Compression** (quantization/pruning [3]),
- **Robustness assessments** (adversarial/noise tolerance [14]), and
- **Hybrid architectures** (Vision Transformers [20]).

The evolution of efficient deep learning techniques will move these extensions further toward maximizing performance even though they maintain both speed and minimal footprint.

REFERENCES

- [1] S. Yang, P. Li, X. Xiong, F. Shen, and J. Zhao, "AdaAugment: A Tuning-Free and Adaptive Approach to Enhance Data Augmentation," arXiv preprint arXiv:2405.11467, May 2024. [Online]
- [2] L. Smets, W. Van Leekwijck, I. J. Tsang, and S. Latré, "An Encoding Framework for Binarized Images Using HyperDimensional Computing," arXiv preprint arXiv:2312.00454, Dec. 2023. [Online]
- [3] F. I. Meem and N. D. R. Mishu, "An Evaluation of Machine Learning Models for Deep Learning Image Classification with Fashion-MNIST Dataset," ResearchGate Preprint, May 2024. [Online]
- [4] S. Bouzidi, G. Hcini, I. Jdey, and F. Drira, "Convolutional Neural Networks and Vision Transformers for Fashion MNIST Classification: A Literature Review," arXiv preprint arXiv:2406.03478, Jun. 2024. [Online]
- [5] A. Gupta, I. Hasan, D. K. Prasad, and D. K. Gupta, "Data-Efficient Training of CNNs and Transformers with Coresets: A Stability Perspective," arXiv preprint arXiv:2303.02095, Mar. 2023. [Online]
- [6] D. Bridgman, B. Lindsay, and S. Shen, "Deep Learning vs. Linear Classifiers on the Fashion-MNIST Dataset," COMP5318 - Machine Learning Data Mining, Assignment Report, May 2019. [Online]
- [7] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [8] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. <https://doi.org/10.3390/make5040087>
- [9] M. Tan and Q. V. Le, "EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks," in *Proc. 36th Int. Conf. Machine Learning (ICML)*, PMLR 97, 2019, pp. 6105–6114. [Online]
- [10] H. M. Abd Alaziz et al., "Enhancing Fashion Classification with Vision Transformer (ViT) and Developing Recommendation Fashion Systems Using DINOVA2," *Electronics*, vol. 12, no. 20, p. 4263, Oct. 2023. <https://doi.org/10.3390/electronics12204263>
- [11] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms," arXiv preprint arXiv:1708.07747, Sep. 2017. [Online]
- [12] C. Löffler and C. Mutschler, "IALE: Imitating Active Learner Ensembles," arXiv preprint arXiv:2007.04637, Sep. 2020. [Online]
- [13] O. Nocentini, J. Kim, M. Z. Bashir, and F. Cavallo, "Image Classification Using Multiple Convolutional Neural Networks on the Fashion-MNIST Dataset," *Sensors*, vol. 22, no. 23, p. 9544, Dec. 2022. <https://doi.org/10.3390/s22239544>
- [14] A. Krizhevsky, I. Sutskever, and G. E. Hinton, "ImageNet Classification with Deep Convolutional Neural Networks," *Advances in Neural Information Processing Systems*, vol. 25, pp. 1097–1105, 2012. <http://dx.doi.org/10.1145/3065386>
- [15] Y. Yang, W. Wang, H. Fu, and C.-C. J. Kuo, "On Supervised Feature Selection from High Dimensional Feature Spaces," arXiv preprint arXiv:2203.11924, Jun. 2022. [Online]
- [16] C.-H. Tseng, X. Zeng, H.-C. Liu, and S.-J. Lee, "Perturbated Gradients Updating within Unit Space for Deep Learning," *IEEE Conference Paper*, 2023. <https://doi.org/10.1109/IJCNN55064.2022.9892245>
- [17] R. I. Mukhamediev, "State-of-the-Art Results with the Fashion-MNIST Dataset," *Mathematics*, vol. 12, no. 20, p. 3174, Oct. 2024. <https://doi.org/10.3390/math12203174>

- [18] H. Touvron, M. Cord, M. Douze, F. Massa, A. Sablayrolles, and H. Jégou, “Training Data-Efficient Image Transformers Distillation Through Attention,” in **Proc. 38th Int. Conf. Machine Learning (ICML)**, PMLR 139, 2021. [Online]
- [19] K. Simonyan and A. Zisserman, “Very Deep Convolutional Networks for Large-Scale Image Recognition,” arXiv preprint arXiv:1409.1556, Apr. 2015. [Online]
- [20] S. Mehta and M. Rastegari, “MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer,” arXiv preprint arXiv:2110.02178, Oct. 2021. [Online]